Ensemble Learning & Support Vector Machines

STAT3612: Statistical Machine Learning

Yinghao Zhu

HKU

November 2026

Outline & Companion Notebook

1. Part 1: Ensemble Learning
2. Part 2: Support Vector Machines (SVM)
3. Part 3: Final Group Project Starter

Hands-On Companion: `ensemble_svm_tutorial.ipynb`

- **Part 1 (Ensemble Learning):** Design philosophy recap, `DecisionTree` depth diagnosis, `Bagging` (`oob_score`), `RandomForest`, `AdaBoost`, `GBDT`, and `XGBoost`.
- **Part 2 (Support Vector Machines):** Margin geometry, `linear`/`poly`/`rbf`/`sigmoid` kernels, (C, γ) sensitivity, and `StandardScaler` pipelines.
- **Part 3 (Group Project Starter):** Multi-table aggregation, `GroupKFold`, and `log_loss` baseline.

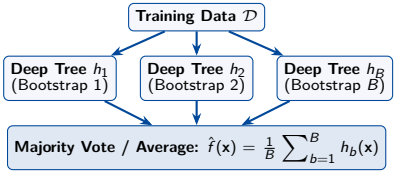
Design Recap: Parallel vs. Sequential Ensembles

Core Motivation

A single deep tree has **low bias, high variance** (memorizes noise); a single shallow tree has **high bias** (coarse steps). Ensembles combine multiple trees $h_1, \ldots, h_B$ via two workflows:

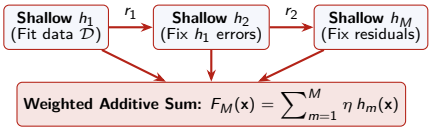
1. Parallel Ensembles (Reduce Variance ↓)

- **Models:** Bagging, Random Forest
- **Workflow:** Train deep trees **independently in parallel** on bootstrap resamples (RF also subsamples features per split), then average:



2. Sequential Ensembles (Reduce Bias ↓)

- **Models:** AdaBoost, Gradient Boosting, XGBoost
- **Workflow:** Train shallow weak learners **sequentially**; each new tree targets prior errors (re-weighted samples or residuals):



Visual Intuition I: Parallel Ensembles (Variance Reduction)

Bootstrap Averaging Mechanism:

- Each deep tree h_b sees a different bootstrap sample ($\approx 63.2\%$ unique rows) and overfits **different noisy points** (first 3 panels).
- Taking the **majority vote across 500 trees** (bottom-right) cancels out random overfitting spikes.
- **Bagging vs. Random Forest:**
  - **Bagging:** Considers all p features per split.
  - **Random Forest:** Samples $\approx \sqrt{p}$ features per split to decorrelate trees.

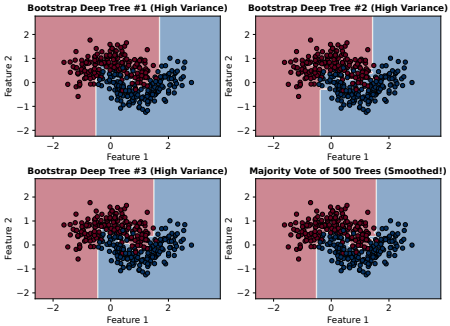


Figure Takeaway: Trees #1–#3 overfit different noise spikes; voting across 500 trees cancels the noise into a smooth boundary.

Visual Intuition II: Sequential Ensembles (Bias Reduction)

Iterative Residual Fitting:

- Boosting starts with a **coarse shallow tree** ($M = 1$, top-left: high bias, only 2–3 splits).
- Subsequent trees fit the **errors of the current ensemble** ($M = 5 \to 20 \to 100$), progressively bending the boundary around the two crescents.
- **Two Error-Targeting Rules:**
  - **AdaBoost:** Up-weights misclassified samples w_i.
  - **GBDT / XGBoost:** Fits new trees to negative gradients (residuals) $-\partial\mathcal{L}/\partial F(\mathbf{x}_i)$.

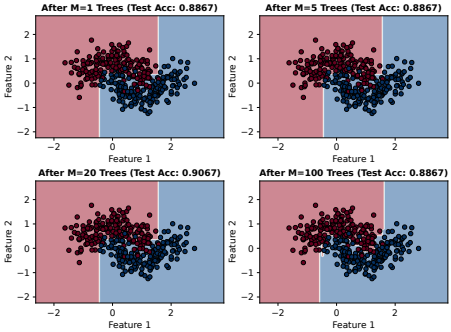


Figure Takeaway: A single shallow tree underfits ($M = 1$); adding residual-correcting trees ($M = 5 \to 20 \to 100$) progressively sculpts the curve.

Part 1 Experimental Roadmap: The `make_moons` Task

1. **Benchmark Task ($N = 500$, `noise` $= 0.3$):**
   - **2D Binary Classification:** Predict $y \in \{0, 1\}$ from $\mathbf{x} = (x_1, x_2) \in \mathbb{R}^2$ across two interlocking crescents.
   - **2D Visualization:** Plotting the full (x_1, x_2) decision surface lets us directly inspect overfitting vs. boundary smoothing.
2. **Controlled Comparison (350 Train / 150 Test):**
   1. **Baseline:** Single Tree (`max_depth=None` vs. 3).
   2. **Parallel:** `Bagging` $\rightarrow$ `Random Forest`.
   3. **Sequential:** `AdaBoost` $\rightarrow$ `GBDT` $\rightarrow$ `XGBoost`.

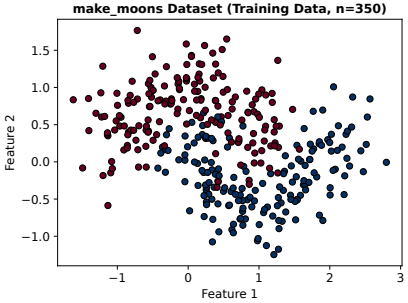


Figure Takeaway: True Bayes boundary is a smooth curve; class overlap in the middle penalizes models that chase noisy points.

Hands-On Setup Code: Data Split & Boundary Plotter

1. Generate & Split `make_moons`:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.inspection import DecisionBoundaryDisplay
X, y = make_moons(
  n_samples=500, noise=0.3, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.3, random_state=42)
```

2. Reusable 2D Boundary Helper:

```
def plot_decision_boundary(clf, X, y, ax, title):
  DecisionBoundaryDisplay.from_estimator(
    clf, X, response_method="predict",
    cmap=plt.cm.RdBu, alpha=0.5, ax=ax)
  ax.scatter(X[:, 0], X[:, 1], c=y,
    cmap=plt.cm.RdBu, edgecolors="k")
  ax.set_title(title)
```

Helper Function Role:

- `DecisionBoundaryDisplay` evaluates `clf.predict` over a 2D grid covering (x_1, x_2).
- Every model calls this helper on `X_train`, `y_train` and evaluates `accuracy_score` on `X_test`, `y_test`.

Baseline: Unrestricted Decision Tree (`max_depth=None`)

Core Code (`DecisionTreeClassifier`):

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# Default max_depth=None grows until leaves are pure
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
tree_accuracy = accuracy_score(y_test, y_pred_tree)
# Train Acc:  1.0000 | Test Acc:  0.8533
```

Key Takeaway:

- Never deploy an unrestricted single tree on noisy data—you must either **prune depth** or **ensemble**.

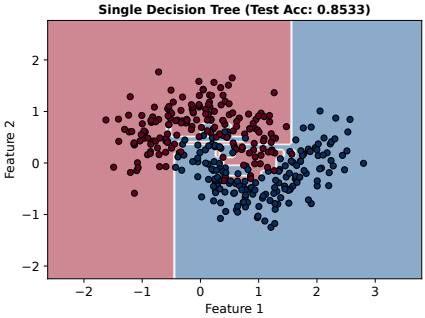


Figure Takeaway: Axis-aligned splits carve thin rectangular islands around noisy points (Train = 1.00 → Test = **0.8533**, severe overfitting).

Diagnostic: Unrestricted vs. Pre-Pruned Tree (max_depth=3)

Pre-Pruning via `max_depth=3`:

```
# Restrict tree depth to regularize capacity
tree_pruned = DecisionTreeClassifier(
  max_depth=3, random_state=42)
tree_pruned.fit(X_train, y_train)
pruned_acc = accuracy_score(
  y_test, tree_pruned.predict(X_test))
# max_depth=None -> Test Acc:  0.8533
# max_depth=3 -> Test Acc:  0.8867
```

- **Takeaway:** Pruning a single tree trades variance for bias; to get a **smooth curve**, we need an ensemble.

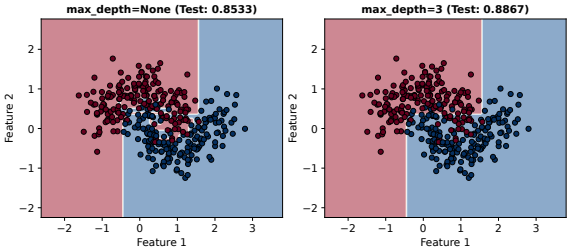


Figure Takeaway: Capping `max_depth` $= 3$ removes noisy pockets (**0.8867**), but restricts the boundary to a blocky 3-step staircase.

Parallel I: Bagging & Out-of-Bag Validation (`BaggingClassifier`)

Core Code (500 Bootstrap Trees):

```
from sklearn.ensemble import BaggingClassifier
bag_clf = BaggingClassifier(
  DecisionTreeClassifier(random_state=42),
  n_estimators=500, max_samples=1.0,
  bootstrap=True, oob_score=True,
  n_jobs=-1, random_state=42)
bag_clf.fit(X_train, y_train)
# OOB Acc:  bag_clf.oob_score_ -> 0.8971
# Test Acc:  accuracy_score(...)  -> 0.8933
```

- **Takeaway:** Pass `oob_score=True` for a free validation estimate (0.8971) from the $\approx 36.8\%$ left-out rows per tree.

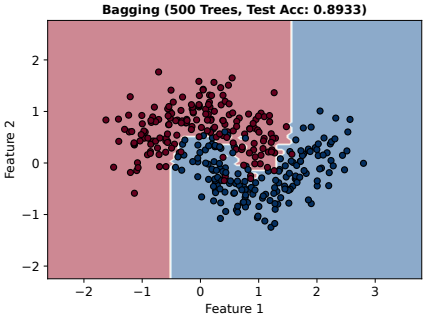


Figure Takeaway: Voting across 500 bootstrap trees erases single-tree islands and forms a smooth curve (Test = **0.8933**, +4.0%).

Parallel II: Random Forest (`RandomForestClassifier`)

Core Code & Feature Importance:

```
from sklearn.ensemble import RandomForestClassifier
rnd_clf = RandomForestClassifier(
  n_estimators=500, max_features="sqrt",
  oob_score=True, n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
rf_acc = accuracy_score(y_test,
rnd_clf.predict(X_test))
# Random Forest Test Acc:  0.9000 (Best!)
# Feature importances:  rnd_clf.feature_importances_
```

- **Takeaway:** Subsampling features (`max_features="sqrt"`) decorrelates trees ($\rho \downarrow$), making RF the strongest **out-of-the-box baseline**.

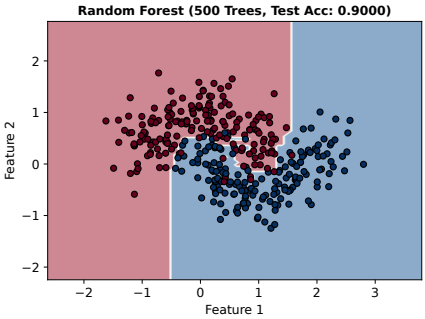


Figure Takeaway: Decorrelated splits produce the cleanest crescent boundary and the highest test accuracy (**0.9000**).

Sequential I: Adaptive Re-Weighting (`AdaBoostClassifier`)

Core Code (200 Decision Stumps):

```
from sklearn.ensemble import AdaBoostClassifier
# Base learner:  depth-1 decision stump
ada_clf = AdaBoostClassifier(
  DecisionTreeClassifier(max_depth=1,
random_state=42),
  n_estimators=200, learning_rate=0.5,
  random_state=42)
ada_clf.fit(X_train, y_train)
ada_acc = accuracy_score(y_test,
ada_clf.predict(X_test))
# AdaBoost Test Acc:  0.8800
```

- **Takeaway:** Re-weighting boosts simple stumps, but **avoid AdaBoost on noisy labels** (exponential outlier weights).

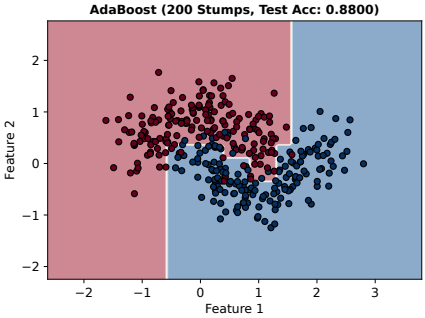


Figure Takeaway: 200 depth-1 stumps approximate the curve (**0.8800**), but up-weighted noisy points create rectangular kinks.

Sequential II: Gradient Boosted Trees (`GradientBoostingClassifier`)

Core Code (Residual Fitting):

```
from sklearn.ensemble import
GradientBoostingClassifier
# Sequential depth-3 trees fit pseudo-residuals
gbrt_clf = GradientBoostingClassifier(
  n_estimators=100, learning_rate=0.1,
  max_depth=3, random_state=42)
gbrt_clf.fit(X_train, y_train)
gbrt_acc = accuracy_score(y_test,
gbrt_clf.predict(X_test))
# Gradient Boosting Test Acc:  0.8867
```

- **Takeaway:** Fitting **negative gradients** of any loss is more flexible and stable than exponential re-weighting.

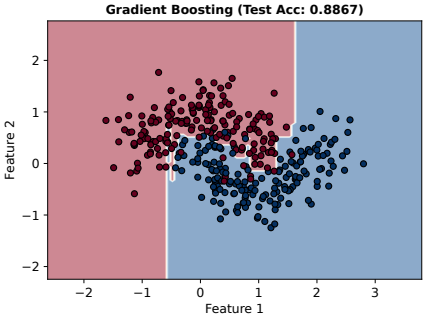


Figure Takeaway: Fitting 100 depth-3 trees to pseudo-residuals eliminates AdaBoost's outlier kinks and lifts test accuracy to **0.8867**.

Sequential III: Scalable Regularized GBDT (XGBoost)

Core Code (xgb.XGBClassifier):

```
import xgboost as xgb
xgb_clf = xgb.XGBClassifier(
  n_estimators=100, learning_rate=0.1, max_depth=3,
  subsample=0.8, colsample_bytree=0.8,
  eval_metric="logloss", random_state=42)
xgb_clf.fit(X_train, y_train)
xgb_acc = accuracy_score(y_test,
xgb_clf.predict(X_test))
# XGBoost Test Acc:  0.8867
```

- **Takeaway:** Prefer `XGBClassifier` on tabular tasks: it adds 2nd-order gradients, leaf L_2 penalty (`reg_lambda`), column subsampling, and native `NaN` routing.

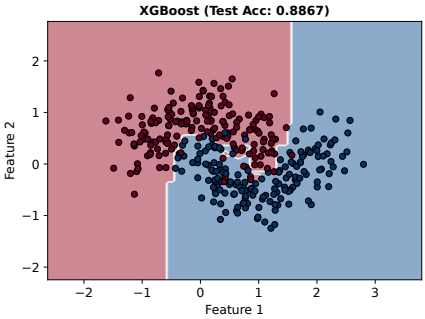


Figure Takeaway: Regularized leaf weights and stochastic subsampling yield a smooth boundary (**0.8867**) with fast parallel training.

Diagnostic: Boosting Hyperparameter Sensitivity

Shrinkage vs. Tree Depth Trade-Off:

```
# Good:  small lr + shallow trees
g_good = GradientBoostingClassifier(
  learning_rate=0.1, max_depth=3,
  n_estimators=100, random_state=42)
# Overfit:  large lr + deep trees
g_over = GradientBoostingClassifier(
  learning_rate=1.0, max_depth=6,
  n_estimators=200, random_state=42)
```

- **Takeaway:** Unlike RF (where more trees is safe), **Boosting overfits if step size or depth is too large**—keep `max_depth` $\in [3, 6]$ and `learning_rate` ≤ 0.1.

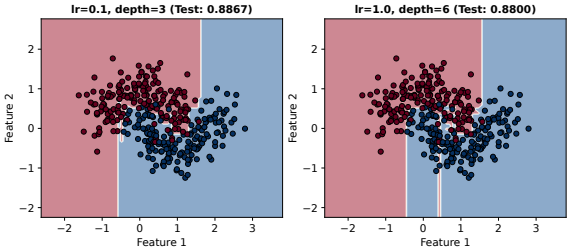


Figure Takeaway: $\texttt{lr} = 0.1, \texttt{depth} = 3$ (left) stays smooth; aggressive $\texttt{lr} = 1.0, \texttt{depth} = 6$ (right) brings back overfitting vertical strips.

Beyond Trees: Ensembling Diverse Model Families (`VotingClassifier`)

Combine Any Models (Linear + Trees + SVM):

```
from sklearn.ensemble import VotingClassifier
voting_clf = VotingClassifier(estimators=[
  ("lr", LogisticRegression(random_state=42)),
  ("rf", rnd_clf), # 500-tree Random Forest
  ("svc", SVC(kernel="rbf", probability=True))],
  voting="soft").fit(X_train, y_train)
# LR: 0.8600 | RF: 0.9000 | SVM: 0.9067
# Soft Voting Ensemble Test Acc:  0.9267 (Best!)
```

- **Hard vs. Soft Voting:** `voting="hard"` takes majority class labels; `voting="soft"` averages predicted probabilities $\hat{p}_m(y = 1 \mid \mathbf{x})$.
- **Takeaway:** Ensembling is **not limited to trees**—blending diverse model families (`LR` + `RF` + `XGBoost` + `SVM`) cancels uncorrelated errors!

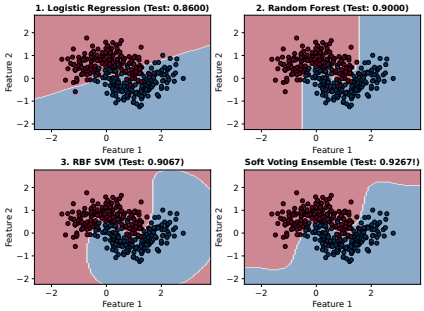


Figure Takeaway: Blending a linear model (**0.8600**), tree ensemble (**0.9000**), and RBF SVM (**0.9067**) lifts test accuracy to **0.9267**.

Part 1 Summary & Assignment 3 (Q1–Q2) Code Pattern

Benchmark Results on `make_moons`:

Model	Test Acc.
Soft Voting (LR + RF + SVM)	**0.9267**
Random Forest (500 Trees)	**0.9000**
Bagging (500 Trees)	0.8933
Pruned Tree (`max_depth` = 3)	0.8867
XGBoost (`max_depth` = 3)	0.8867
Gradient Boosting (`max_depth` = 3)	0.8867
AdaBoost (200 Stumps)	0.8800
Unrestricted Tree (`None`)	0.8533

Assignment 3 (Q1–Q2) Template:

```
from sklearn.tree import plot_tree
# Q1(c-d):  Entropy split + tree visualization
dt = DecisionTreeClassifier(
 max_depth=3, criterion="entropy")
dt.fit(X_train, y_train); plot_tree(dt, filled=True)
# Q2(a-b):  Sweep max_depth = 2..12
for d in range(2, 13):
 t = DecisionTreeClassifier(
  max_depth=d, random_state=2025)
 rf = RandomForestClassifier(n_estimators=80,
  max_features=4, max_depth=d, random_state=2025)
```

SVM Design Recap I: Maximum Margin, Support Vectors & C

Geometric Intuition:

- SVM fits a hyperplane $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = 0$ (solid line) with maximum margin width $\frac{2}{\|\mathbf{w}\|}$ between dashed gutters $f(\mathbf{x}) = \pm 1$.
- **Support Vectors (hollow rings):** Only points on or inside the street ($y_i f(\mathbf{x}_i) \leq 1$) determine $\mathbf{w}$; far-away points have zero influence.
- **Role of Penalty C:**
  - **Small C** (left): Wide street, tolerates outliers.
  - **Large C** (right): Narrow street, strictly penalizes margin errors.

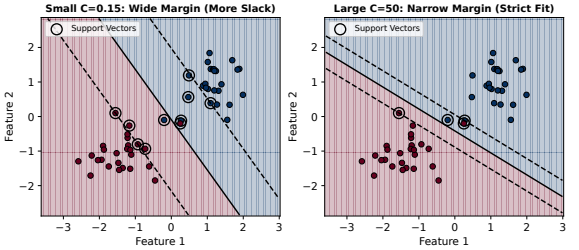


Figure Takeaway: Small $C = 0.15$ (left) keeps a wide, robust margin street; large $C = 50$ (right) forces a narrow street to avoid slack errors.

SVM Design Recap II: The Kernel Trick (2D $\rightarrow$ 3D Lifting)

From Non-Linear 2D to Linear 3D:

- **Problem (left):** Concentric rings in 2D (x_1, x_2) cannot be separated by any straight line.
- **Lifting Map (right):** Adding $z = x_1^2 + x_2^2$ lifts the outer ring upward so a flat linear plane slices cleanly between the classes.
- **The Kernel Trick:**

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$$

computes high-dimensional inner products **directly from 2D inputs**, without explicitly constructing $\phi(\mathbf{x})$.

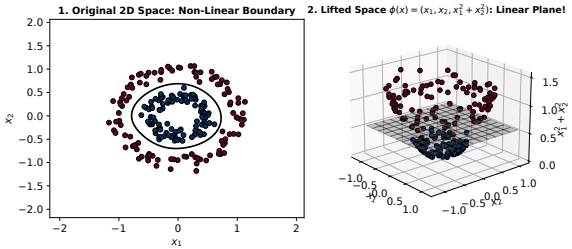


Figure Takeaway: Non-separable 2D rings (left) become linearly separable by a flat plane after lifting to 3D via $z = x_1^2 + x_2^2$ (right).

Part 2 Experimental Roadmap: `Iris` 2D & `XOR` Benchmarks

Two Complementary 2D Classification Tasks:

1. **Task A — Botanical Species (`Iris`, $N = 100$):**
   - **Task:** Classify 1: Versicolor vs. 2: Virginica from **petal length** and **petal width** (cm).
   - **Goal:** Test how 4 kernels (`linear`, `poly`, `rbf`, `sigmoid`) and (C, γ) generalize on 25 test flowers.
2. **Task B — 4-Quadrant `XOR` Stress Test ($N = 300$):**
   - **Task:** Opposite quadrants share a label ($\text{sign}(x_1) \neq \text{sign}(x_2)$), where any linear boundary fails ($\approx 50\%$).

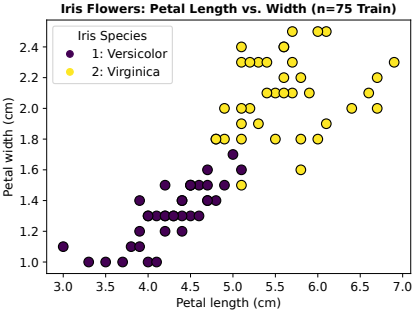


Figure Takeaway: *Versicolor* (purple) and *Virginica* (yellow) overlap near $(4.8, 1.6)$ cm, testing margin behavior on noisy boundaries.

Hands-On Setup Code: `Iris` Split & Margin Contour Plotter

1. Select 2 Features & 2 Species from Iris ($N = 100$):

```
data = load_iris()
X = data.data[:, [2, 3]] # petal length & width
y = data.target
idx = np.where((y == 1) | (y == 2)) # classes 1 & 2
X, y = X[idx], y[idx]
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.25, stratify=y)
```

2. Plot Boundary (0), Margins (± 1) & Support Vectors:

```
clf = svm.SVC(kernel=kernel, gamma=2).fit(X, y)
DecisionBoundaryDisplay.from_estimator(clf, X, ax=ax,
  response_method="decision_function",
  plot_method="contour", levels=[-1, 0, 1],
  colors=["k", "k", "k"], linestyles=["-", "-", "-"])
ax.scatter(clf.support_vectors_[:, 0],
  clf.support_vectors_[:, 1],
  s=150, facecolors="none", edgecolors="k")
```

Plot Legend:

- **Solid line ($f(\mathbf{x}) = 0$):** Decision boundary.
- **Dashed lines ($f(\mathbf{x}) = \pm 1$):** Margin gutters.
- **Hollow rings (`clf.support_vectors_`):** Support vectors on or inside the margin.

Task A (Part 1): Linear vs. Polynomial Kernel on `Iris`

1. Linear Kernel (Test Acc = 0.88):
$K(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{x}_2$

```
mmc_clf = SVC(kernel="linear", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.88
```

2. Polynomial Kernel (Test Acc = 0.84):
$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \mathbf{x}_1^T \mathbf{x}_2 + r)^d$

```
mmc_clf = SVC(kernel="poly", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.84
```

- **Takeaway:** Benchmark `kernel="linear"` first before higher-degree polynomials.

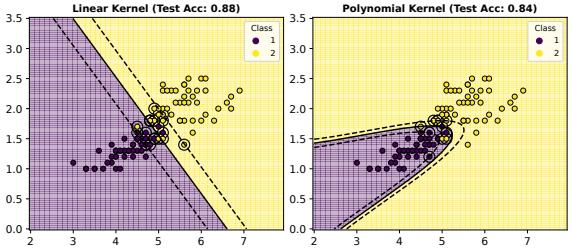


Figure Takeaway: Linear (left, **0.88**) keeps a wide parallel margin; degree-3 Poly (right, **0.84**) pinches the margin too tightly and loses accuracy.

Task A (Part 2): RBF vs. Sigmoid Kernel on `Iris`

3. RBF / Gaussian Kernel (Test Acc = 0.92):
$K(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2\right)$

```
mmc_clf = SVC(kernel="rbf", random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.92 (Best!)
```

4. Sigmoid Kernel (Test Acc = 0.48):
$K(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\gamma \mathbf{x}_1^T \mathbf{x}_2 + r)$

```
mmc_clf = SVC(kernel="sigmoid",
random_state=42)
mmc_clf.fit(X_train, y_train)
mmc_clf.score(X_test, y_test) # 0.48
```

- **Takeaway:** Use `rbf` (default in `SVC`) and `linear` as primary choices; avoid `sigmoid`.

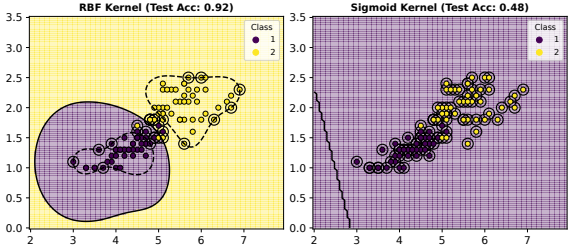


Figure Takeaway: RBF (left, **0.92**) curves smoothly around the species boundary; Sigmoid (right, **0.48**) produces degenerate contours.

Diagnostic: Tuning C and gamma in RBF SVM

How (C, γ) Shape the Boundary:

```
# Smooth margin:  moderate gamma & C
svc_smooth = SVC(
  kernel="rbf", gamma=0.5, C=1.0).fit(X, y)
# Overfit islands:  large gamma & C
svc_overfit = SVC(
  kernel="rbf", gamma=30, C=50.0).fit(X, y)
```

- **Takeaway:** Large γ shrinks the Gaussian radius $1/\sqrt{\gamma}$; always tune (C, γ) jointly via log-grid CV to prevent island overfitting.

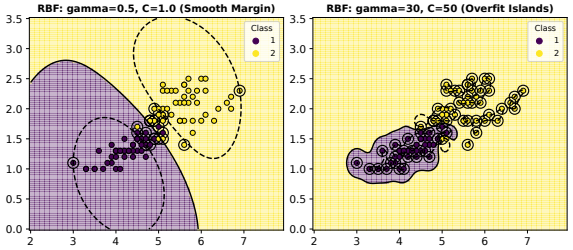


Figure Takeaway: Moderate $\gamma = 0.5$, $C = 1$ (left) is smooth; $\gamma = 30$, $C = 50$ (right) wraps tiny islands around individual flowers.

Task B: The 4-Quadrant XOR Non-Linear Stress Test

Core Code (Alternating Quadrants):

```
np.random.seed(0)
X = np.random.randn(300, 2)
# Class 1 when x1 and x2 have opposite signs
y = np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)
_, ax = plt.subplots(2, 2, figsize=(8, 8))
args = dict(long_title=False,
support_vectors=False)
for k, a in zip(
  ["linear", "poly", "rbf", "sigmoid"], ax.flat):
  plot_training_data_with_decision_boundary(
    k, a, **args)
```

- **Takeaway:** When feature interactions are non-monotonic (like XOR), `kernel="rbf"` is essential.

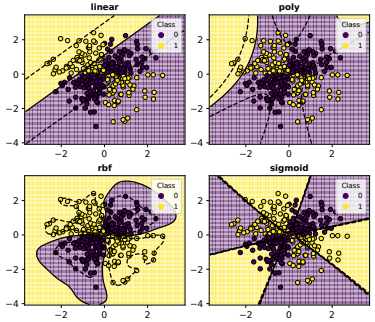


Figure Takeaway: Linear, Poly, and Sigmoid fail on alternating quadrants; only **RBF** (bottom-left) encloses all four quadrants cleanly.

Practical SVM Pipeline: Feature Scaling & Assignment 3 (Q4)

Mandatory Engineering Rule for SVM:

- **Trees / Ensembles:** Split rules $x_j \leq t$ only depend on rank ordering $\Longrightarrow$ scale-invariant.
- **SVM:** Margin $\|\mathbf{w}\|^2$ and RBF distance $\|\mathbf{x}_1 - \mathbf{x}_2\|^2$ are scale-sensitive; unscaled features with large numeric ranges dominate the kernel!
- **Takeaway:** Always standardize features (`StandardScaler`) fit on `X_train` before `SVC`.

Assignment 3 (Q4) Template:

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix
# Q4(a-b): Scale features & fit RBF SVC
scaler = StandardScaler()
X_tr = scaler.fit_transform(X_train)
X_te = scaler.transform(X_test)
svc = SVC(C=1.5, kernel="rbf").fit(X_tr, y_train)
# Q4(c-d): Support vectors & confusion matrix
n_sv = len(svc.support_vectors_)
cm = confusion_matrix(y_test, svc.predict(X_te))
```

Group Project Starter I: Multi-Table Features & `GroupKFold`

Task Recap: Predicting a Short-Term Learning-Platform Pause

Given Days 0–28 logs (`train.csv`, `vle_events.parquet`, `past_assessments.parquet`), predict probability $P(\texttt{target} = 1)$ of inactivity on Days 29–42, evaluated by **Binary Log Loss**.

```
# 1. Aggregate event logs to sample_id level
vle_agg = vle.groupby("sample_id").agg(
  total_clicks=("clicks", "sum"),
  active_days=("day", "nunique")).reset_index()
df = train.merge(vle_agg, on="sample_id", how="left")
# 2. GroupKFold prevents student_group leakage!
gkf = GroupKFold(n_splits=5)
for tr, va in gkf.split(
  X, y, groups=df["student_group"]):
  clf.fit(X.iloc[tr], y.iloc[tr])
  p_val = clf.predict_proba(X.iloc[va])[:, 1]
  score = log_loss(y.iloc[va], p_val)
```

Two Critical Pitfalls to Avoid:

- **Leakage:** Repeated students share `student_group`; always split by group rather than random `train_test_split`.
- **Metric:** Log loss requires calibrated probabilities (`predict_proba[:, 1]`), never hard 0/1 labels (`predict`).

Group Project Starter II: Applying Today's Models & Next Steps

Plugging In RF, XGBoost & SVM Baselines:

```
# 1. Tree ensembles handle mixed tabular scales
rf = RandomForestClassifier(
  n_estimators=300, max_depth=8, min_samples_leaf=10)
xgb_m = xgb.XGBClassifier(
  n_estimators=300, learning_rate=0.05, max_depth=4)
# 2. SVM needs imputation + scaling + Platt
svm_pipe = make_pipeline(SimpleImputer(),
  StandardScaler(), SVC(C=1.0, probability=True))
# 3. Export clipped probabilities to submission.csv
sub["prediction"] = np.clip(
  xgb_m.predict_proba(X_te)[:, 1], 1e-6, 1 - 1e-6)
```

Directions for Your Team:

- **Temporal Dynamics:** Compare Week 4 clicks (Days 22–28) vs. Weeks 1–3 to capture recent drop-off trends.
- **Assessment Behavior:** Late submission rate (`submitted_day` > `due_day`) and weighted completion.
- **Calibration & Ensembling:** Blend `XGBoost` + `RandomForest` probabilities to reduce log loss.

Take-Home Summary: Ensembles & SVM Cheat Sheet

Method	Mechanism	Primary Effect	Key `sklearn` Args	Scale?
Decision Tree	Recursive axis splits	Low bias, high var.	`max_depth`, `criterion`	No
Bagging	Parallel bootstrap trees	Reduces variance	`n_estimators`, `oob_score=True`	No
Random Forest	Bagging + random features	Decorrelates trees	`n_estimators`, `max_features`	No
AdaBoost	Sequential re-weighted stumps	Reduces bias	`n_estimators`, `learning_rate`	No
GBDT / XGBoost	Sequential trees on residuals	Reduces bias	`max_depth=3`, `learning_rate`	No
Linear SVM	Max-margin hyperplane	Global linear margin	`kernel="linear"`, `C`	**Yes**
Kernel SVM (RBF)	Margin in $\phi(\mathbf{x})$ space	Smooth non-linear	`kernel="rbf"`, `C`, `gamma`	**Yes**

Next Steps

Run `ensemble_svm_tutorial.ipynb` locally for hands-on experiments, **Assignment 3** (due Nov 26), and your **Final Group Project** starter pipeline.